

```

CU_ASSERT(is_even(3) == 0);
}

int
main (void)
{
    CU_pSuite pSuite = NULL;

    /* Initialize CUnit test registry */
    if (CUE_SUCCESS != CU_initialize_registry())
        return CU_get_error();

    /* Add suite to registry */
    pSuite = CU_add_suite("Basic_Test_Suite", init_suite,
clean_suite);
    if (NULL == pSuite) {
        CU_cleanup_registry();
        return CU_get_error();
    }

    /* add test to suite */
    if ((NULL == CU_add_test(pSuite, "test_is_even", test_is_
even)))
    {
        CU_cleanup_registry();
        return CU_get_error();
    }

    /* Run tests using Basic interface */
    CU_basic_run_tests();

    /* Clean up registry and return */
    CU_cleanup_registry();
    return CU_get_error();
}

{.bash}
$ gcc basic.c -o basic -lcunit -lcurses

```

The above step can also be abstracted and automated in a Makefile for subsequent compilation and testing, as follows:

```

{.make}
CC = gcc
CUNIT_LDFLAGS = -lcunit -lcurses

objects = basic

all:
    ${foreach file,$(objects), $(CC) $(file).c -o $(file)
$(CUNIT_LDFLAGS) ;)

clean:

```

```
rm -f *- *.o $(objects)
```

You can type 'make' in the terminal to compile the code, as follows:

```

{.bash}
$ make
gcc basic.c -o basic -lcunit -lcurses ;

```

On execution, you will get the following output:

```

{.bash}
$ ./basic
CUnit - A unit testing framework for C - Version 2.1-2
http://cunit.sourceforge.net/

```

Run Summary:	Type	Total	Ran	Passed	Failed	Inactive
	suites	1	1	n/a	0	0
	tests	1	1	1	0	0
	asserts	3	3	3	0	n/a

Elapsed time = 0.000 seconds

The above example uses the Basic interface output from CUnit. A number of assertions have been defined in the CUnit/CUnit.h file. Some of them are:

CU_ASSERT(int expression)	Check expression is non-zero
CU_ASSERT_FATAL(int expression)	
CU_ASSERT_TRUE(value)	Check value is non-zero
CU_ASSERT_TRUE_FATAL(value)	
CU_ASSERT_FALSE(value)	Check value is zero
CU_ASSERT_FALSE_FATAL(value)	
CU_ASSERT_EQUAL(actual, expected)	Check actual == expected
CU_ASSERT_EQUAL_FATAL(actual, expected)	
CU_ASSERT_NOT_EQUAL(actual, expected)	Check actual != expected
CU_ASSERT_NOT_EQUAL_FATAL(actual, expected)	
CU_ASSERT_PTR_NULL(value)	Check pointer == NULL
CU_ASSERT_PTR_NULL_FATAL(value)	
CU_ASSERT_PTR_NOT_NULL(value)	Check pointer != NULL
CU_ASSERT_PTR_NOT_NULL_FATAL(value)	